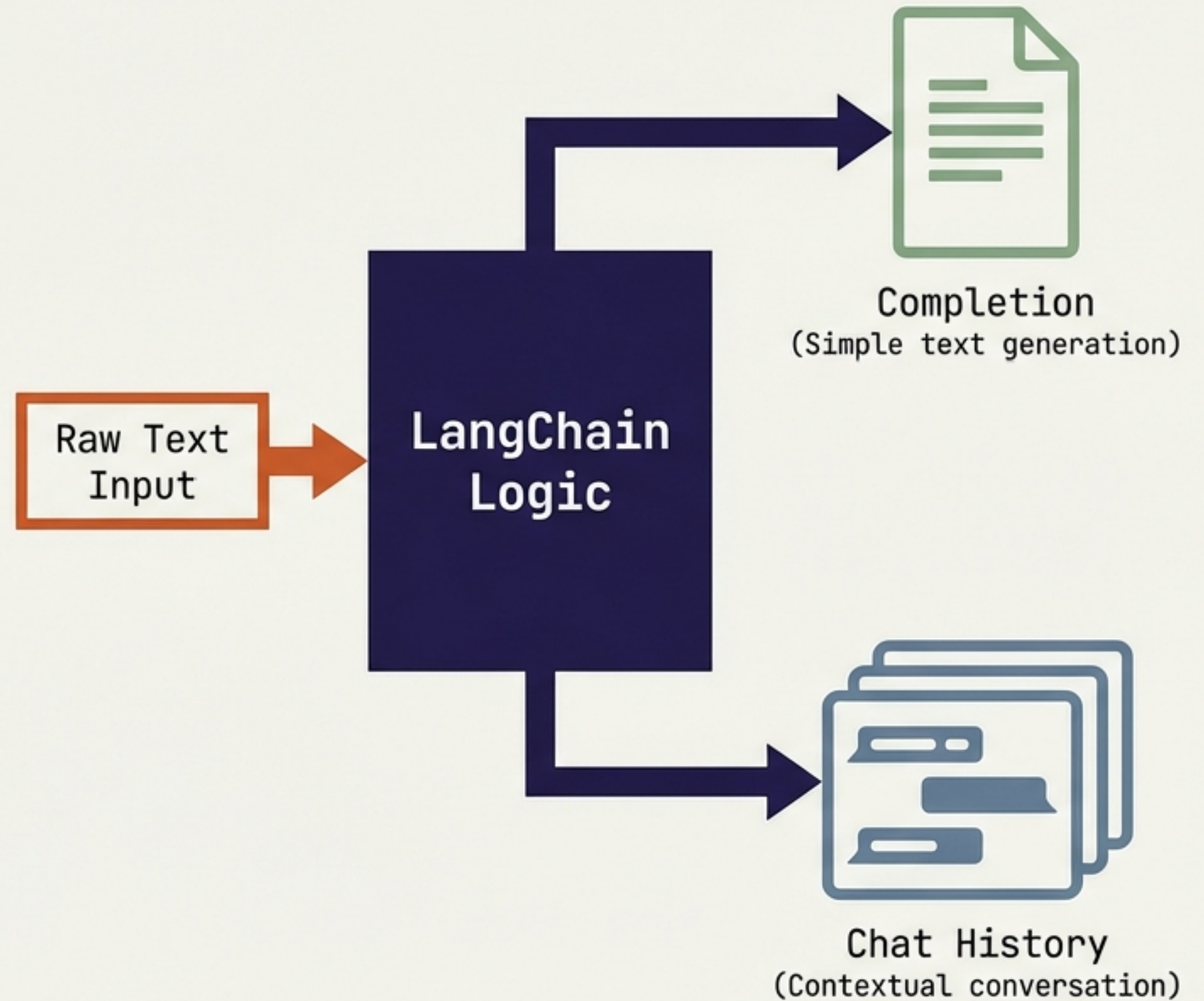


Mastering Prompts in LangChain

From Static String Inputs to Dynamic, Contextual Conversations



Based on the technical tutorial by CampusX

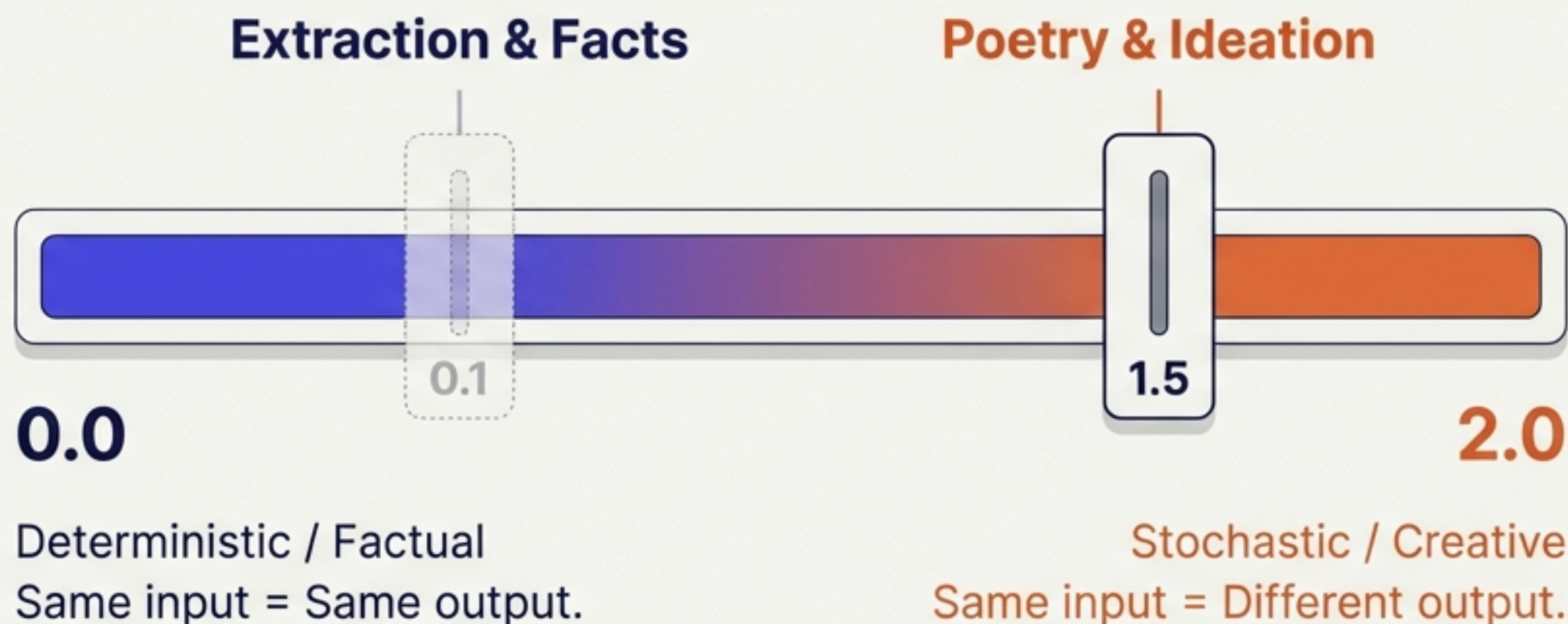
THE CORE CONCEPT: SENSITIVITY & TEMPERATURE

What is a Prompt?

The input layer sent to the LLM. While multimodal (image/video) is possible, this deck focuses on text.

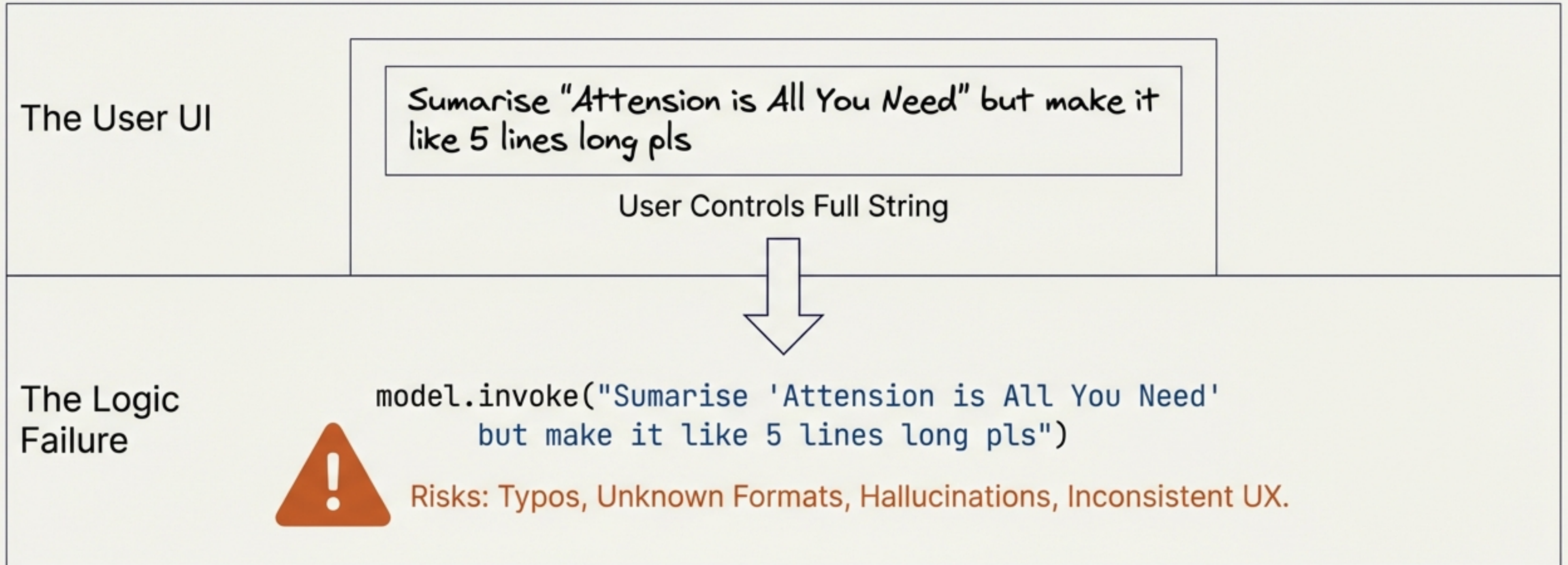
The Butterfly Effect

LLMs are highly sensitive. Slight phrasing changes = massive output changes. This necessitates 'Prompt Engineering'.



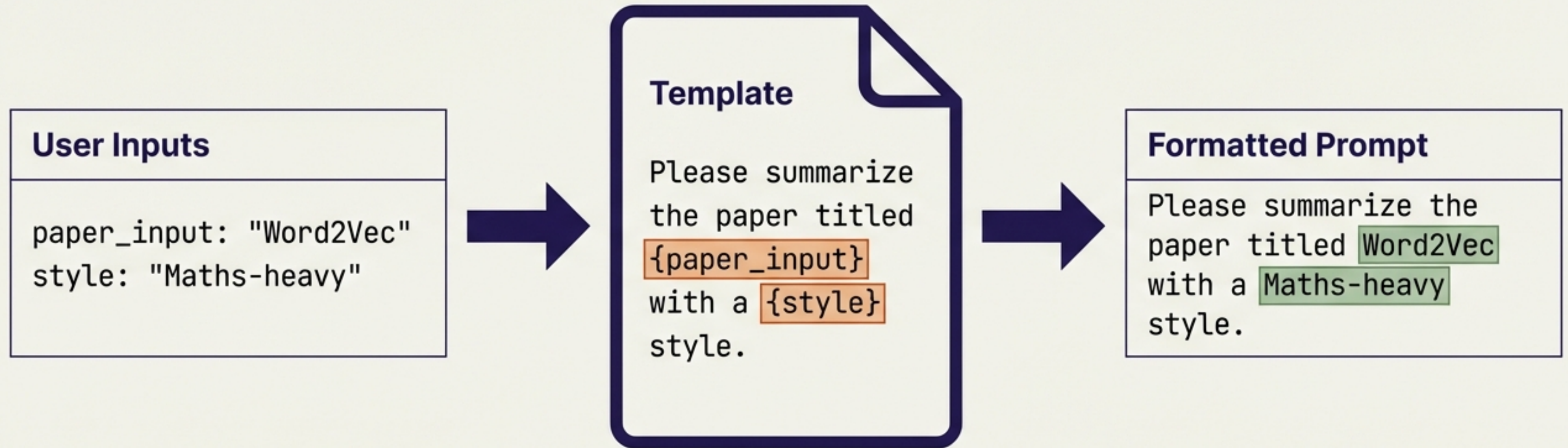
THE TRAP OF STATIC PROMPTS

Scenario: Research Paper Assistant



Giving users control over the **entire** prompt string creates a fragile application. Structure must be separated from input.

CONTROLLED INPUTS WITH PROMPTTEMPLATE



```
template = PromptTemplate(input_variables=['paper_input', 'style'], template=template_str)
```

```
template.invoke({'paper_input': 'Word2Vec', 'style': 'Maths-heavy'})
```


Why Not Just Use Python f-strings?

Why `PromptTemplate` is safer than `f'Summarize {paper}'`



Input Validation

Automatically checks for missing variables before the API call. Prevents runtime crashes and wasted tokens.

```
ValidationError: 'style' missing
```



Serialization

Save templates as JSON files to decouple logic from code. Load prompts dynamically across different app modules.

```
template.save('prompt.json')
```

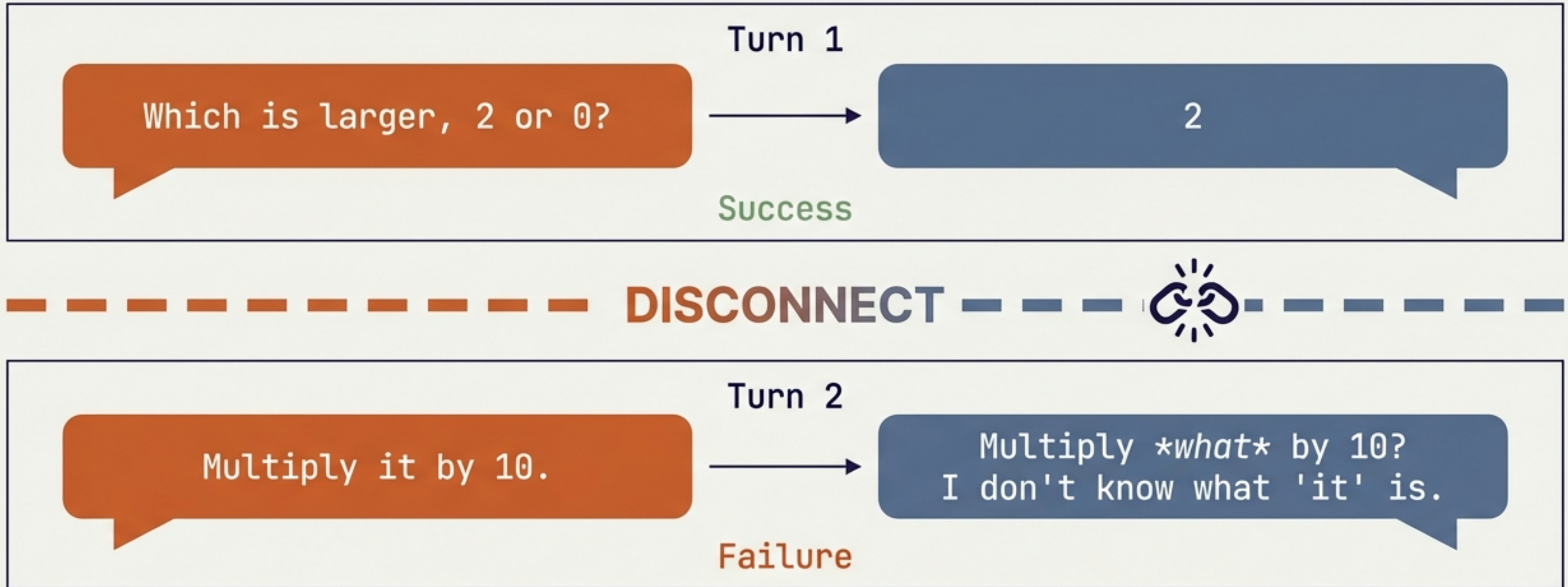


LangChain Integration

Natively fits into LCEL Chains. Allows piping templates directly into models.

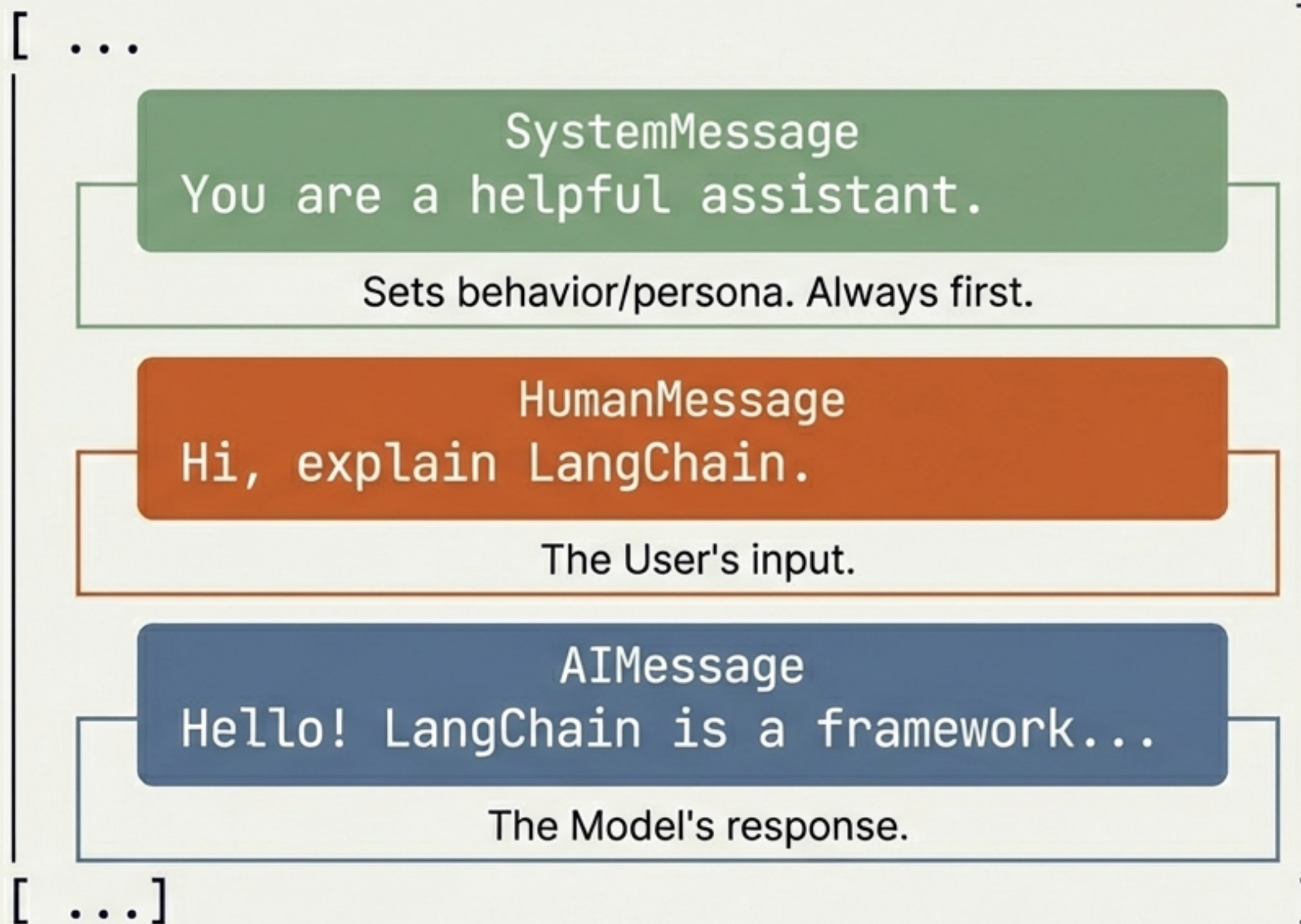
```
chain = template | model
```


THE MULTI-TURN CHALLENGE: CONTEXT & MEMORY



LLMs are **stateless (Amnesic)**. To solve this, we must send the **entire conversation history** back to the model with every new request.

The Anatomy of LangChain Messages



```
from langchain_core.messages  
import SystemMessage,  
HumanMessage,  
AIMessage
```

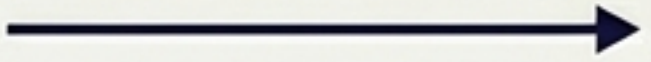

“ChatPromptTemplate”: Templating for Conversations

Used when specific roles or messages in a chat flow need to be dynamic.

Define Structure

```
prompt = ChatPromptTemplate.  
    from_messages([  
        ("system", "You are a helpful  
{domain} expert."),  
        ("human", "Explain {topic}.")  
    ])
```

`.invoke({'domain':
'Cricket',
'topic': 'Doosra'})`



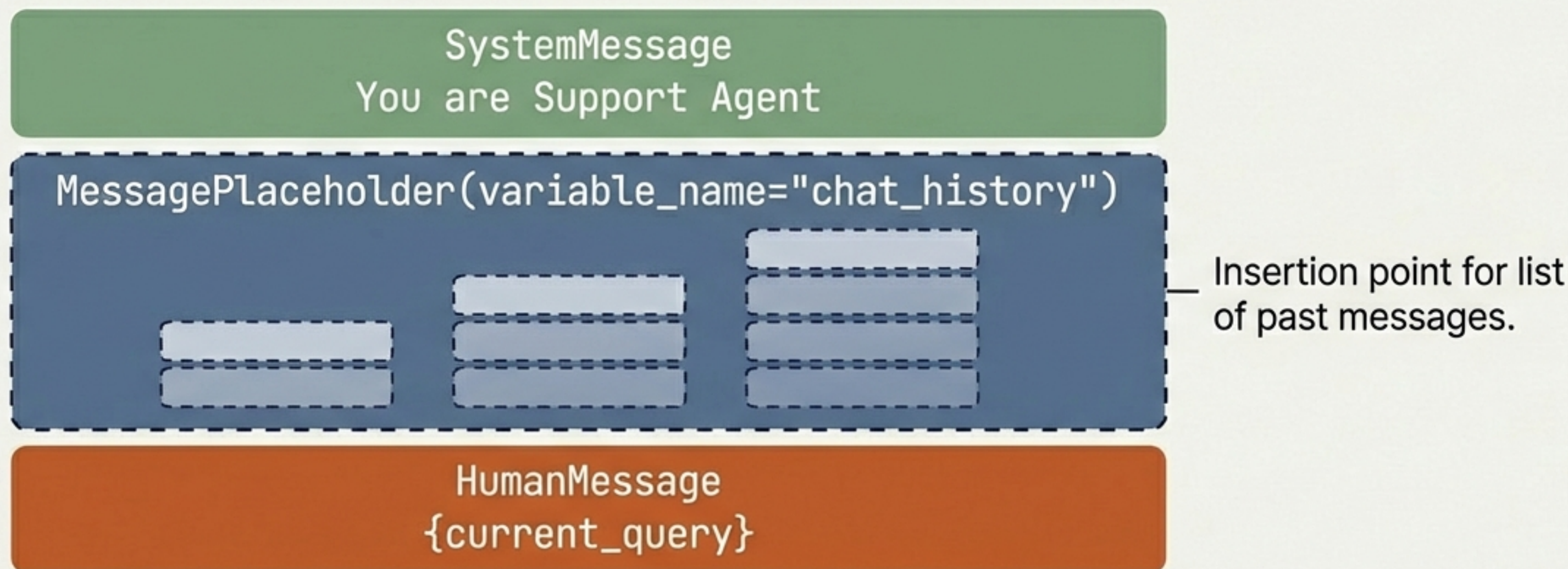
Resulting Message List

SystemMessage
You are a helpful
Cricket expert.

HumanMessage
Explain Doosra.

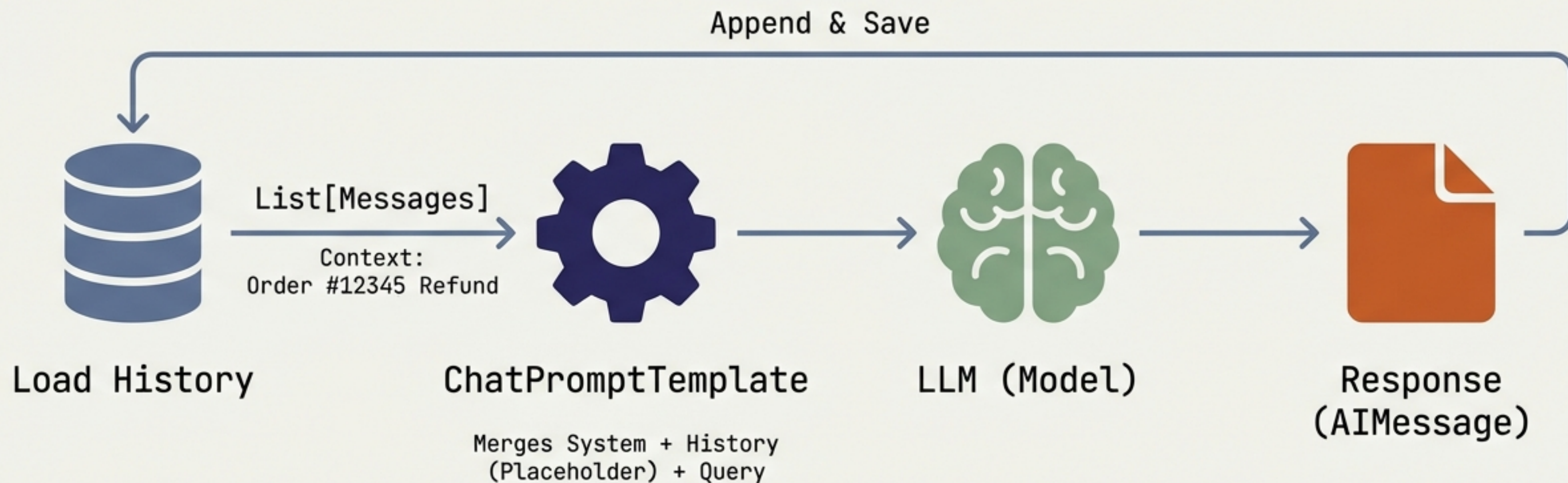
Dynamic History with `MessagePlaceholder`

Scenario: Amazon Refund Support Chat



Why: Allows injecting an arbitrary number of past messages into a structured template.

Putting It Together: The Chat Workflow



Summary of Key Components



PromptTemplate

For single-turn, string-based inputs.
Use for standalone tasks like
summarization.



ChatPromptTemplate

For multi-turn, list-based inputs.
Structures conversations with
distinct roles.



Messages

``System``, ``Human``, ``AI``. The standard
schema for context data.



MessagePlaceholder

Injects dynamic lists (history) into
templates. Essential for memory.

Beyond Structure: Next Steps

Prompt Construction (This Deck)

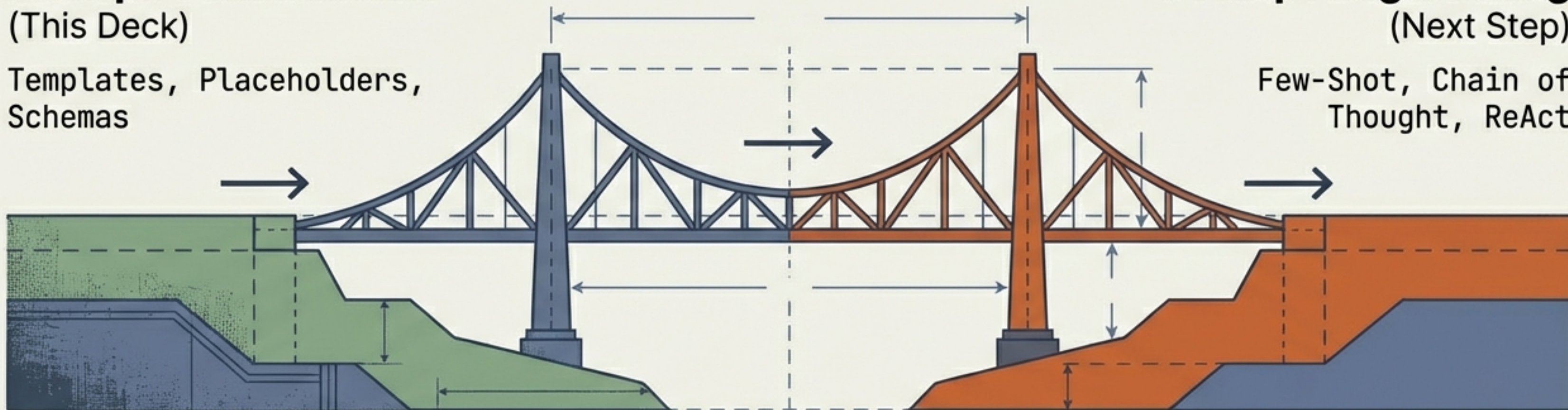
(This Deck)

Templates, Placeholders,
Schemas

Prompt Engineering (Next Step)

(Next Step)

Few-Shot, Chain of
Thought, ReAct



**“Mastering the classes gives you the vehicle;
mastering the prompt text gives you the destination.”**